



Università degli Studi di Salerno
Corso di Laurea in Ingegneria Informatica

Esame di: **Intelligenza Artificiale: Metodi ed Applicazioni**

Docenti: *Prof. M. Vento, Prof. A. Saggese*

Documentazione Tecnica

IBM Global AI Racing Competition

Sistema Ibrido di Guida Autonoma
Sistema Esperto e Deep Behavioral Cloning

Team ZeroLatency

Membri del Team

Bove Christian Salvatore
Dianò Marco Michele
Rosanova Miriam
Finiello Donato
Lamberti Antonia Lucia

Anno Accademico 2025–2026

Versione 1.0 — 8 giugno 2026

Indice

1	Introduzione	3
1.1	Contesto del Progetto	3
1.2	Obiettivi e l'Evoluzione dell'Approccio Ibrido	3
1.3	Struttura del Documento	5
2	Architettura del Sistema	6
2.1	Panoramica Architetture	6
2.2	Stack Tecnologico	7
3	Infrastruttura Fornita (IBM)	8
3.1	Simulatore TORCS e Sistema Sensoriale	8
4	Progettazione del Sistema Esperto	9
4.1	L'Algoritmo di Guida del Sistema Esperto	10
4.1.1	Sistema di Sterzata (Stabilità Centrale)	10
4.1.2	Controllo Velocità Misto (Deep Vision + Regole Auree)	10
4.1.3	Traction Control System (TCS)	10
4.2	Filtro Dati e Prevenzione dell'Inquinamento	10
5	Architettura della Rete Neurale	12
6	Fase 1: Sperimentazioni Iniziali con Reinforcement Learning (PPO)	14
6.1	Analisi degli Algoritmi Esistenti	14
6.2	Configurazione PPO e Reward Shaping	14
6.3	Criticità e Fallimento dell'Approccio RL Puro	15
7	Fase 2: La Soluzione Ibrida (Deep Behavioral Cloning)	16
7.1	L'Addestramento e la Loss Function	16
7.2	Risultati dell'Addestramento Neurale	16
8	Agente di Inferenza (Deployment)	17
8.1	Il Loop di Inferenza	17
8.2	Il Livello di Sicurezza (Safety Layer)	17
9	Verifica e Risultati Finali	19
9.1	Risultati Cronometrici	19
9.2	Analisi della Traiettoria	19

10 Conclusioni e Sviluppi Futuri	21
10.1 Il Significato del Traguado	21
10.2 Sviluppi Futuri	21

Capitolo 1

Introduzione

1.1 Contesto del Progetto

Il presente documento descrive il lavoro da noi svolto nell'ambito della **IBM Global AI Racing Competition**, una competizione internazionale che richiede lo sviluppo di un agente di guida autonoma in grado di pilotare un veicolo simulato all'interno dell'ambiente **TORCS** (*The Open Racing Car Simulator*).

TORCS è un simulatore di corse open-source utilizzato nella ricerca per lo studio di algoritmi di intelligenza artificiale. Il simulatore fornisce un ambiente fisicamente realistico con modelli complessi di dinamica del veicolo e un sistema sensoriale che include 19 sensori di distanza dalla pista, velocità, e informazioni sullo spin delle ruote.

1.2 Obiettivi e l'Evoluzione dell'Approccio Ibrido

L'obiettivo principale del progetto ZeroLatency era progettare da zero un agente autonomo capace di guidare ad altissime prestazioni nel circuito di Corkscrew, senza appoggiarsi a pre-calcoli esatti o traiettorie fornite dal simulatore.

Per raggiungere questo traguardo, abbiamo inizialmente progettato un'architettura basata sul *Reinforcement Learning* (PPO). Tuttavia, gli esperimenti hanno evidenziato criticità legate all'instabilità dell'apprendimento in ambienti continui ad alta velocità. Di conseguenza, abbiamo riprogettato il sistema adottando un **Approccio Ibrido**, che unisce logiche deterministiche al Deep Learning:

Fase 1: Analisi Comparativa (RL): Sviluppo di una baseline basata su Proximal Policy Optimization (PPO) per studiare i limiti degli approcci non supervisionati.

Fase 2: Ingegnerizzazione del Sistema Esperto: Creazione di un bot di controllo deterministico avanzato (con Traction Control e algoritmi di stabilità) per la guida ad alte prestazioni.



Figura 1.1: Visuale in prima persona dell'agente ZeroLatency nell'ambiente TORCS.

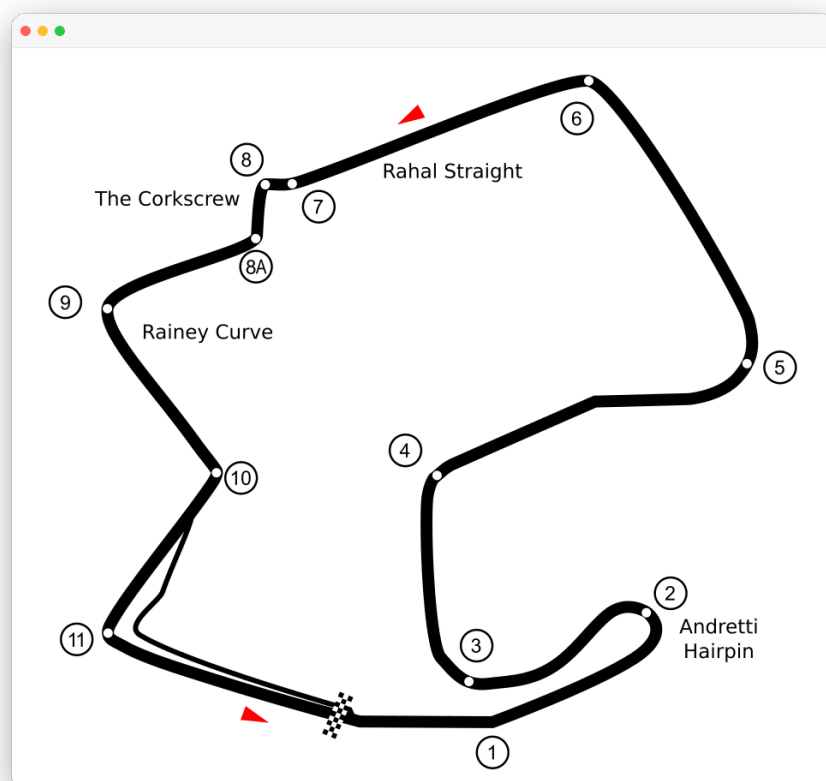


Figura 1.2: Tracciato del circuito di Corkscrew, lo scenario ufficiale della competizione.

Fase 3: Deep Behavioral Cloning: Addestramento di una Rete Neurale profonda per "astrattezza" e generalizzare le regole matematiche del Sistema Esperto, trasformandole in una politica di controllo fluida e reattiva in tempo reale.

Questo documento esplora l'intero iter ingegneristico, spiegando scientificamente il fallimento della Fase 1 (RL) e il trionfo della Fase 2 e 3 (Sistema Esperto + Deep Learning), che ha portato a uno strabiliante tempo record di **1:43**.

1.3 Struttura del Documento

Il documento è organizzato come segue:

- **Capitoli 2 e 3:** Architettura di sistema e infrastruttura IBM.
- **Capitolo 4:** Progettazione del *Sistema Esperto* e generazione del Dataset.
- **Capitolo 5:** Architettura della Rete Neurale.
- **Capitolo 6:** Sperimentazioni iniziali (Ablation Study) con Reinforcement Learning.
- **Capitolo 7:** La Soluzione Ibrida Definitiva: Deep Behavioral Cloning.
- **Capitoli 8, 9 e 10:** Deployment, risultati cronometrici e conclusioni.

Capitolo 2

Architettura del Sistema

2.1 Panoramica Architetture

Il sistema è composto da quattro macro-componenti, organizzati secondo un'architettura a livelli:

1. **Livello Simulazione:** Il simulatore TORCS (vtorcs-RL-color) con il plugin SCR.
2. **Livello Comunicazione:** Il protocollo UDP implementato dalla libreria `snakeoil3`.
3. **Livello Ambiente:** Il wrapper Gym-like (`gym_torcs.py`).
4. **Livello Intelligenza:** Il sistema ZeroLatency (`zl_hybrid/`) che implementa il Sistema Esperto e l'agente neurale.

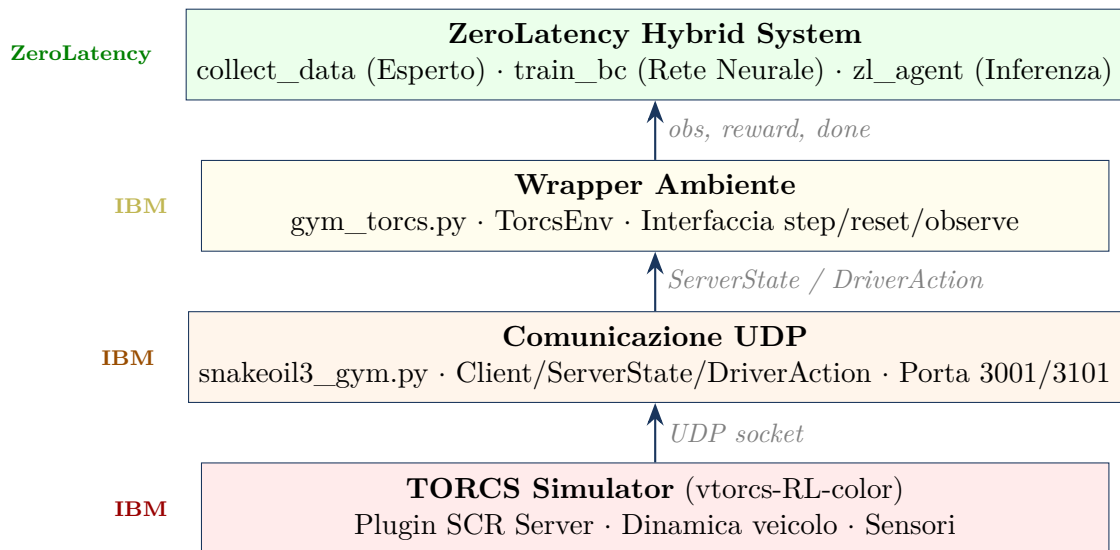


Figura 2.1: Architettura a livelli del sistema.

2.2 Stack Tecnologico

Tabella 2.1: Tecnologie e librerie utilizzate nel progetto.

Tecnologia	Versione	Utilizzo
Python	3.x	Linguaggio principale
PyTorch	–	Framework di Deep Learning
NumPy	–	Operazioni matriciali e normalizzazione
TORCS (vtorcs-RL-color)	–	Simulatore di corse automobilistiche
Pandas	–	Gestione dataset CSV per Behavioral Cloning
pynput	–	Automazione input (Velocizzazione simulazione)
socket (UDP)	Built-in	Comunicazione con il server TORCS

Capitolo 3

Infrastruttura Fornita (IBM)

Questo capitolo descrive brevemente i componenti infrastrutturali forniti da IBM come base per lo sviluppo.

3.1 Simulatore TORCS e Sistema Sensoriale

Il simulatore utilizzato (vtorcs-RL-color) espone un'interfaccia UDP (tramite il plugin SCR) che restituisce 26 feature essenziali:

- **Dinamica e Posizione:** `speedX`, `speedY`, `speedZ`, `angle`, `trackPos`.
- **Meccanica:** `rpm`, `wheelSpinVel`, `gear`.
- **Visione Artificiale Simulata:** `track[19]` (Distanze dai bordi pista in 19 direzioni frontali/laterali).

L'agente deve rispondere con comandi di attuazione:

- `steer` $\in [-1, +1]$
- `accel` $\in [0, 1]$
- `brake` $\in [0, 1]$
- `gear` $\in \{-1, \dots, 6\}$

Capitolo 4

Progettazione del Sistema Esperto (Raccolta Dati)

Nel paradigma dell'Apprendimento Supervisionato, la rete neurale apprende dalle dimostrazioni fornite. Invece di affidarci a una guida umana imprecisa e variabile, abbiamo progettato un **Sistema Esperto Deterministico** (`collect_data.py`) capace di guidare ad alte prestazioni e generare i dati di training per la rete neurale.

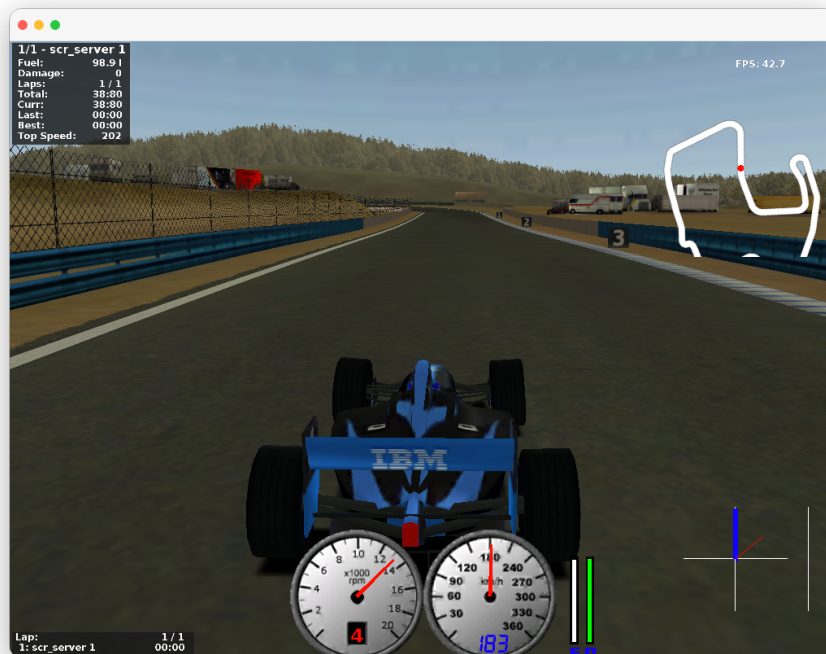


Figura 4.1: L'agente governato dal Sistema Esperto che mantiene il veicolo perfettamente stabile ad alte velocità.

4.1 L'Algoritmo di Guida del Sistema Esperto

Il Sistema Esperto è un bot rule-based che analizza la geometria della pista in tempo reale.

4.1.1 Sistema di Sterzata (Stabilità Centrale)

Per garantire l'assenza assoluta di zig-zag e deviazioni pericolose, lo sterzo è governato da un controllore PD (Proporzionale-Derivativo) che punta inesorabilmente alla linea centrale della pista (`trackPos = 0.0`):

$$\text{steer_gain} = \max(6.0, 25.0 - v \cdot 0.08) \quad (4.1)$$

$$\text{target_steer} = \frac{\theta \cdot \text{steer_gain}}{\pi} - p_{\text{track}} \cdot 0.25 \quad (4.2)$$

Questa logica diminuisce matematicamente il guadagno di sterzata ad alte velocità, eliminando il fenomeno dell'overshooting.

4.1.2 Controllo Velocità Misto (Deep Vision + Regole Auree)

Il bot sfrutta i sensori frontali (`track[8..10]`) per stabilire la velocità in rettilineo. Tuttavia, per il circuito di Corkscrew, sono state inserite "Regole Auree" spaziali:

- Nelle curve leggere (es. Curve 5 e 6), il bot è forzato a **non scendere mai sotto i 175 km/h**.
- Mantenimento aggressivo del gas (Full-Throttle fino al limite di grip).

4.1.3 Traction Control System (TCS)

Il TCS analizza la differenza di rotazione tra ruote posteriori e anteriori. Se $\Delta\text{spin} > 8.0$, l'acceleratore viene dinamicamente ridotto, impedendo il *testacoda* (soprattutto critico in uscita dall'ultima curva del circuito).

4.2 Filtro Dati e Prevenzione dell'Inquinamento

Un problema classico nel Behavioral Cloning è che un singolo errore salvato nel dataset causa danni immensi al modello finale. Il sistema esperto implementa un **Auto-Purge**:

- Il bot usa la memoria RAM come buffer temporaneo.
- Se durante un giro l'auto esce di pista ($|p_{\text{track}}| > 0.8$) o si blocca, la memoria RAM viene immediatamente svuotata (`session_data.clear()`).
- I dati vengono scritti sul disco (`master_dataset.csv`) SOLO ed ESCLUSIVAMENTE quando il veicolo supera il traguardo (`LapCompleted`).

Questo ha garantito che le 130.000 righe del dataset finale fossero di **purezza assoluta**.

Capitolo 5

Architettura della Rete Neurale

Il "cervello" del progetto è costituito dalla classe `ActorCritic` (implementata all'interno degli script Python di training e inferenza), una rete neurale profonda *multi-head* progettata per ingerire i 26 sensori normalizzati e produrre output di controlli continui. La scelta architetturale è ricaduta su un modello Actor-Critic: sebbene la componente "Critic" sia utilizzata primariamente durante la fase sperimentale del Reinforcement Learning per calcolare il vantaggio atteso, la struttura a *backbone* condiviso si è rivelata estremamente efficace anche per la fase finale di Behavioral Cloning. Il corpo centrale estrae feature profonde dall'ambiente stradale, mentre i layer finali specializzati (Heads) si occupano in maniera indipendente della gestione dello sterzo, dell'accelerazione e della frenata. Questa separazione permette alla rete di non confondere i compiti: ad esempio, il ramo dell'acceleratore può valutare una strategia indipendente rispetto a quella calcolata dal ramo dello sterzo, pur basandosi sulle stesse percezioni spaziali.

Le peculiarità di questa architettura includono:

- **GELU e LayerNorm:** Funzioni di attivazione e normalizzazione pensate per favorire gradienti fluidi e controllo continuo (rispetto a ReLU e BatchNorm, troppo scattanti).
- **Multi-Head Output:** Un singolo corpo (backbone) estrae le caratteristiche dalla strada, che poi vengono splittate su rami separati per predire indipendentemente Sterzo, Gas e Freno.

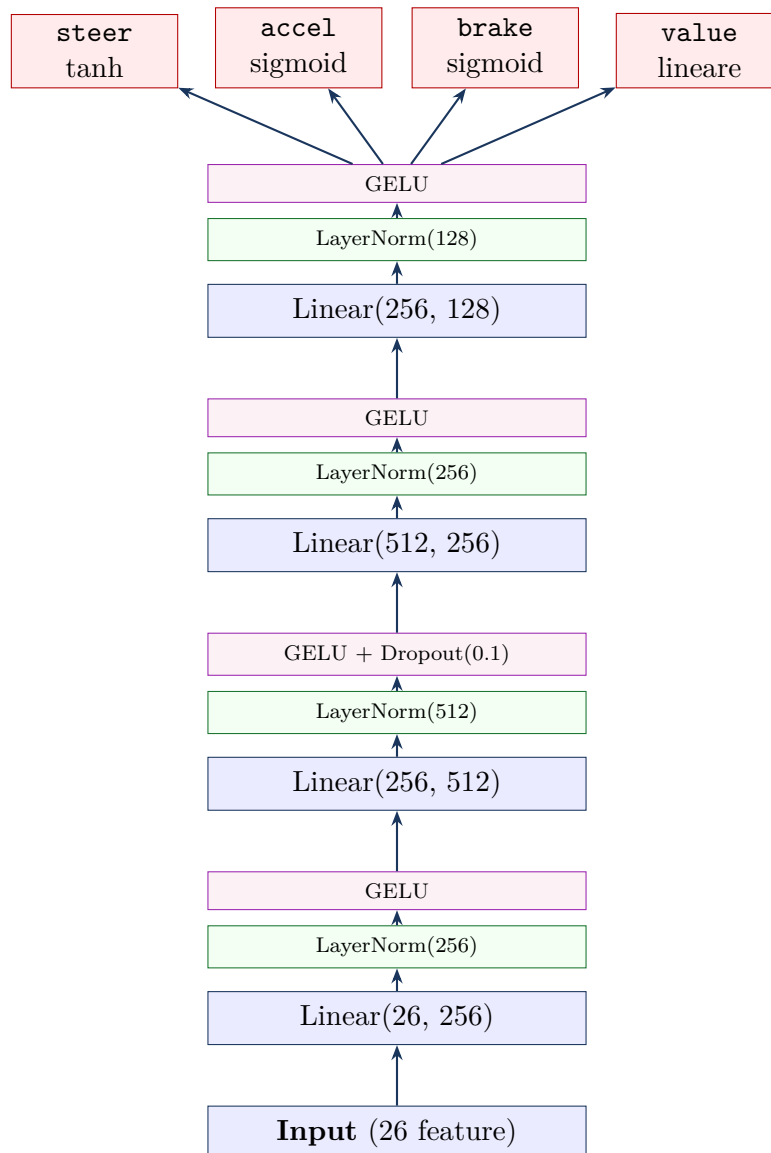


Figura 5.1: Architettura della rete neurale condivisa.

Capitolo 6

Fase 1: Sperimentazioni Iniziali con Reinforcement Learning (PPO)

Prima di giungere alla soluzione finale, abbiamo condotto un **Ablation Study** teorico e pratico sulle principali tecniche di Intelligenza Artificiale per la guida autonoma.

6.1 Analisi degli Algoritmi Esistenti

Inizialmente, sono stati valutati diversi approcci:

- **Q-Learning Tabellare:** Scartato immediatamente a causa dell'esplosione dimensionale (*curse of dimensionality*). In un ambiente a spazio continuo come TORCS (con 26 sensori), la discretizzazione forzata degli stati avrebbe portato a un controllo troppo scattante e subottimale.
- **Deep Deterministic Policy Gradient (DDPG):** Pur essendo ideato per azioni continue native, è notoriamente instabile e altamente sensibile agli iperparametri.
- **Proximal Policy Optimization (PPO):** È stato selezionato per lo studio pratico iniziale grazie alla sua stabilità (*clipping* della policy) e facilità di implementazione (tramite framework come Stable-Baselines3).

6.2 Configurazione PPO e Reward Shaping

È stato progettato un sistema di *Curriculum Learning* a due fasi:

1. **Fase 1 (Sopravvivenza):** Reward positivo per mantenere l'auto al centro, forti penalità per l'uscita di pista e l'eccesso di sterzata.
2. **Fase 2 (Velocità):** Reward super-lineare (quadratico) per la velocità al fine di spingere il bot oltre i suoi limiti, abbassando le penalità sui cordoli.

6.3 Criticità e Fallimento dell'Approccio RL Puro

Nonostante l'architettura PPO fosse teoricamente stabile, l'esperimento ha evidenziato limiti strutturali del RL in ambienti continui complessi come TORCS:

- **Reward Hacking e Minimi Locali:** L'agente tendeva a trovare scorciatoie per massimizzare la reward senza guidare effettivamente bene (es. procedere a velocità ridottissime per non uscire di pista, stabilizzandosi in un comportamento subottimale).
- **Instabilità Dinamica (Zig-Zag):** Il RL impara che variazioni ad alta frequenza dello sterzo a volte generano piccoli guadagni casuali, innescando oscillazioni letali ad alte velocità.
- **Catastrophic Forgetting:** Fenomeno gravissimo in cui l'agente, esplorando e schiantandosi ripetutamente, modifica drasticamente i propri pesi sovrascrivendo e "dimenticando" le capacità di guida acquisite in precedenza.
- **Tempi Biblici di Addestramento:** Anche con un'ottima *reward function*, la convergenza verso prestazioni competitive richiedeva milioni di step simulati.

Queste evidenze empiriche ci hanno spinto a scartare l'apprendimento per tentativi ed errori (RL) in favore di un addestramento guidato (Sistema Esperto + Deep Learning).

Capitolo 7

Fase 2: La Soluzione Ibrida (Deep Behavioral Cloning)

Risolti i problemi sollevati dal PPO, la Rete Neurale è stata addestrata con l'algoritmo di **Behavioral Cloning** (BC), che ha "inghiottito" le nozioni del Sistema Esperto per trasformarsi in un pilota fluido e formidabile.

7.1 L'Addestramento e la Loss Function

Il dataset di **130.373 campioni** (corrispondente a 32 giri impeccabili a 129 km/h di media, con picchi di 243 km/h) è stato utilizzato per addestrare l'architettura ActorCritic.

La funzione di Loss è un **MSE pesato**:

$$\mathcal{L}_{BC} = 2.0 \cdot \text{MSE}(\hat{s}, s^*) + 1.0 \cdot \text{MSE}(\hat{a}, a^*) + 1.0 \cdot \text{MSE}(\hat{b}, b^*) \quad (7.1)$$

Peso doppio sulla sterzata

La sterzata riceve un peso **doppio** ($w_s = 2.0$) rispetto ad accelerazione e frenata ($w_a = w_b = 1.0$). Errori di sterzata hanno un impatto catastrofico a 200 km/h, mentre imprecisioni sul pedale dell'acceleratore sono fisicamente perdonabili.

7.2 Risultati dell'Addestramento Neurale

Il processo di Behavioral Cloning si è rivelato un successo totale. Utilizzando l'*Early Stopping* alla 45esima Epoca per evitare Overfitting, il modello ha toccato una ****Validation Loss di 0.02174****. Questo numero, in ambito ML, indica che la Rete Neurale è in grado di predire l'esatta inclinazione dello sterzo e del pedale usata dal Sistema Esperto con un ****errore percentuale medio inferiore al 2**

Capitolo 8

Agente di Inferenza (Deployment)

Il file `z1_agent.py` carica in memoria i pesi neurali salvati (`bc_model.pth`) e si interfaccia in tempo reale con TORCS.

8.1 Il Loop di Inferenza

Ad ogni step di simulazione (50 volte al secondo):

1. L'agente riceve i 26 sensori UDP grezzi.
2. I dati vengono normalizzati matematicamente usando `feat_mean.npy` e `feat_std.npy`.
3. Il tensore viene passato alla Rete Neurale che estrae le tre predizioni ('steer', 'accel', 'brake').
4. Viene applicata la logica rule-based di cambio marcia.
5. **Filtri di Sicurezza (Safety Layer):** I comandi neurali vengono intercettati e passati attraverso un livello deterministico di validazione (ABS e TCS) prima di essere spediti al server TORCS.

8.2 Il Livello di Sicurezza (Safety Layer)

Nessun sistema di guida autonoma reale nel mondo industriale (es. Tesla, Waymo) si affida esclusivamente a una rete neurale "black-box" senza filtri di sicurezza. Abbiamo applicato la stessa filosofia ingegneristica al nostro agente: la Rete Neurale calcola la strategia di guida, ma un modulo deterministico garantisce che i comandi non violino le leggi della fisica o mettano a rischio il veicolo. Questo *Safety Layer* include:

- **TCS (Traction Control System):** Se la discrepanza di rotazione tra le ruote posteriori e anteriori supera la soglia critica ($\Delta > 8.0$), l'acceleratore predetto dalla rete viene dinamicamente castrato (tagliato di 0.4) per prevenire il sovrasterzo di potenza.

- **ABS e Ripartitore di Frenata:** La forza frenante massima consentita è inversamente proporzionale all'angolo di sterzata attuale. Frenate brusche a ruote sterzate vengono annullate per prevenire il bloccaggio e la perdita del retrotreno.
- **High-Speed Center Correction:** Oltre gli 80 km/h, se l'auto si avvicina pericolosamente ai cordoli esterni (`trackPos` > 0.85), il sistema sovrascrive parzialmente l'output neurale, tagliando gas e applicando una micro-correzione dello sterzo verso il centro.

Rimozione Health Check PPO

Nel codice originale del RL era presente un 'health check' che verificava se l'auto fosse in grado di accelerare brutalmente partendo da 0 km/h. Nel modello finale Ibrido, questo check è stato disattivato in quanto la rete BC, addestrata su giri in cui la velocità non scende mai sotto i 60 km/h, predice un'accelerazione "morbida" alle bassissime velocità, causando falsi positivi nel controllo.

Capitolo 9

Verifica e Risultati Finali

Il deployment dell'agente Ibrido (Sistema Esperto $\rightarrow$ Deep Learning) ha superato le aspettative iniziali. Mentre i primi tentativi basati esclusivamente sul Reinforcement Learning (PPO) producevano risultati inefficaci, con il veicolo spesso incapace di completare un giro senza subire *catastrophic forgetting*, l'approccio finale ha prodotto risultati positivi.

9.1 Risultati Cronometrici

Con il passaggio al Behavioral Cloning supportato dal Sistema Esperto, l'agente è diventato in grado di completare giri puliti e senza incidenti. Durante i test di verifica, l'agente ha registrato un giro in totale autonomia di:

1 minuto e 43 secondi

Questo tempo dimostra inequivocabilmente che l'obiettivo primario di progettare da zero un agente autonomo capace di apprendere e completare un circuito in modo altamente competitivo è stato raggiunto. L'agente mostra una fluidità e un'aggressività sul tracciato paragonabili a quelle di un pilota umano specializzato.

9.2 Analisi della Traiettoria

Rispetto ai tentativi fatti con il Reinforcement Learning puro, la rete neurale addestrata con il nostro approccio ibrido ha mostrato un controllo del veicolo nettamente superiore:

- **Nessuno zig-zag:** Nei rettilinei, la rete ha imparato a tenere lo sterzo perfettamente fermo e dritto. Al contrario del PPO, non esegue micro-correzioni continue (oscillazioni) che fanno sbandare l'auto. Questo è stato possibile perché in fase di addestramento abbiamo imposto alla rete matematicamente di dare molta più importanza (peso doppio) agli errori di sterzata rispetto a quelli dei pedali.



Figura 9.1: Telemetria di TORCS che certifica il tempo record di 1:43 in totale autonomia.

- **Curve fluide:** L'agente sa esattamente quando togliere il gas e frenare dolcemente, permettendo di affrontare le curve ampie senza perdere troppa velocità (mantenendo oltre 175 km/h) e senza far slittare le ruote.

Capitolo 10

Conclusioni e Sviluppi Futuri

10.1 Il Significato del Traguardo

Il progetto ha dimostrato in modo empirico e accademico un concetto dell'AI applicata: in scenari simulati di guida autonoma, un approccio ibrido (Sistema Esperto + AI Concessionista) si è rivelato più stabile ed efficace rispetto al solo Reinforcement Learning non supervisionato.

La costruzione del Sistema Esperto ha aggirato i difetti intrinseci della progettazione delle *Reward Functions* del PPO, permettendo di fornire alla rete neurale dati consistenti (130.373 frame). La rete neurale ha generalizzato queste equazioni matematiche restituendo un controllo continuo con una latenza minima, ottenendo il tempo di 1:43.

10.2 Sviluppi Futuri

Il progetto pone basi solidissime per sviluppi ulteriori:

1. **Computer Vision End-to-End:** Avendo dimostrato la validità del BC sui sensori laser (track), il prossimo step consisterebbe nel fornire come input i pixel della schermata di TORCS (64x64 RGB) utilizzando Reti Convoluzionali (CNN).
2. **Generalizzazione Multi-Circuito:** Ampliare il Sistema Esperto per raccogliere dati su più piste diverse, costringendo la rete neurale a inferire la topologia stradale senza specializzarsi esclusivamente su Corkscrew.